

Eval-First Agentic RCA: A Deterministic Accuracy/MTTR Regression Gate and Shadow-to-Active Rollout for Shipping LLM-in-the-Loop Reliability Tooling

Victor Robin
BlueRobin (independent homelab research)

June 2026

Abstract

We describe the evaluation methodology that underpins the BlueRobin Debug Agent, an LLM-in-the-loop root-cause-analysis (RCA) service that runs on a self-hosted homelab under a hard ~ 50 EUR/month budget. The methodology is *eval-first*: before any reasoning component (graph blame-propagation, change-point detection, hypothesis-state machine, episodic memory) is allowed to govern production behaviour, it must pass a deterministic offline replay harness that scores the *real* RCA path against a labelled ground-truth incident set and emits an Accuracy@1 (**AC@1**) plus mean-time-to-resolution (MTTR/TTM) scorecard. A per-commit **regression gate** with explicit tolerances (**AC@1** drop ≤ 0.02 , MTTR rise ≤ 30 s) blocks any merge that degrades localisation accuracy or inflates analysis latency. The harness has two arms: a *scorer-only* arm where the labelled cause is planted as the unambiguous top-1 (a regression sentinel, **AC@1** = 1.0 by construction) and an honest *live-path* arm that injects **distractor candidates** with higher topology criticality but lower error-rate, so **AC@1** < 1.0 is achievable on the production ranking path. New components ship behind a **shadow-to-active** rollout (`RcaLiveActivation.Mode` $\in$ {Shadow, Active}): they compute and log their would-be decisions for an observation period before they are permitted to govern. We are explicit that the ground-truth set is small (9 self-seeded incidents spanning ≥ 3 fault classes), that MTTR figures are sub-second replay-clock proxies rather than production MTTR, and that the authors themselves flag the by-construction **AC@1** = 1.0 as an “oversimplified benchmark.” The contribution is not a new accuracy record; it is a reproducible discipline — change-correlation as the primary signal, a deterministic merge gate, and staged activation — that makes the rest of the system measurable and safe to ship.

1 Introduction

Automated RCA for microservice and Kubernetes systems has converged on LLM agents that interleave reasoning and tool calls (ReAct; Yao et al., 2022) over observability backends. The appeal is obvious: an agent can query traces, logs, metrics, deploy history and source code, then narrate a hypothesis. The problem is equally obvious once one tries to ship it: **an LLM-only RCA loop has no native notion of being right, and no native notion of regressing**. A prompt tweak, a model swap, or a new tool can silently degrade localisation accuracy, and an operator would never know, because the only feedback signal is a human reading a plausible-sounding paragraph during an incident.

The independent literature is blunt about the ceiling. The best agent on OpenRCA reaches $\sim 11.3\%$ exact-match [Xu et al., 2025]; the best frontier model on IBM’s ITBench-AA reaches $\sim 47\%$ precision@full-recall [Artificial Analysis and IBM Research, 2026]. Benchmarks themselves

are now suspected of overstating capability: simple rule-based methods match “SOTA” on several of them [Anonymous, 2025]. In this environment, the responsible engineering posture is **advisory / human-in-the-loop** — a human always merges the fix — and the responsible *development* posture is **eval-first**: the eval comes before the feature, and every commit is a regression test (HolmesGPT runs 150+ evals per commit; Robusta and Microsoft, 2025, Eval-Driven Development, 2025).

This paper documents how the BlueRobin Debug Agent operationalises that posture. The system is real, implemented in .NET 10 over FalkorDB and Qdrant, and governed by requirement FEAT-027 (Stack-Graph RCA) extending FEAT-008 (Observability & Auto-Debug). Our contributions:

- **A deterministic offline replay harness** (RcaEvalHarness) that replays a labelled ground-truth incident set through the *actual* production RCA path and emits a per-incident and aggregate AC@1 + MTTR scorecard, byte-for-byte reproducible (NFR-SG-5).
- **A per-commit regression gate** (RcaEvalRegressionGate) with explicit, committed tolerances (AC@1 drop ≤ 0.02 , MTTR rise ≤ 30 s) wired into CI as a merge blocker.
- **An honest live-path arm** that injects distractor candidates — siblings with higher topology criticality but lower error-rate — so that AC@1 = 1.0 is *not* guaranteed and the ranker’s accuracy on the production path is actually measured.
- **A shadow-to-active rollout discipline** (RcaLiveActivation, ADR-039) under which the verification gate, BARO scorer, evidence soft-cap and incident-memory recall each run in *shadow* (compute + log, do not govern) before promotion to *active*.
- **Change-correlation as the primary ranking signal**, motivated by the Google SRE finding that $\sim 70\%$ of outages follow a change to a live system [Beyer et al., 2016].
- **An honest threats-to-validity treatment**: we state the small-N, self-seeded, replay-clock limitations plainly rather than burying them.

2 Related Work

Eval-driven development for agents. The discipline of treating evals as per-commit regression tests, not occasional report cards, is crystallised by the HolmesGPT project (Robusta + Microsoft; CNCF Sandbox), which runs 150+ evals on every commit, and by the broader “the eval comes first” movement [Robusta and Microsoft, 2025, Eval-Driven Development, 2025]. Datadog’s Bits AI SRE similarly scores against a labelled set with an LLM-judge and surfaces an agent trace [Datadog, 2026]. Our harness adopts this discipline but constrains the per-commit oracle to be **deterministic and LLM-free** so it can gate merges cheaply and reproducibly; the LLM-judge is relegated to an optional, scheduled, token-capped path.

Benchmarks for RCA. OpenRCA [Xu et al., 2025] provides the most honest “how bad are LLMs at RCA” number ($\sim 11.3\%$ exact-match over 335 cases). AIOpsLab [Chen et al., 2025] offers 48 problems across four levels with TTD/TTM, step-count and token-cost metrics over live Kubernetes (DeathStarBench) with ChaosMesh faults. ITBench [Jha et al., 2025] introduces a Normalised Topology-Aware Metric and reports MTDD 72 s / MTTR 282 s, and shows traces are load-bearing (removing them drops diagnosis $13.8\% \rightarrow 9.5\%$). RCAEval [Pham et al., 2024] is the cleanest reproducible baseline set — 735 cases, 11 fault types, 15 baselines — and standardises

the AC@1, Avg@k, MRR metric family we adopt. Crucially, the Fault-Propagation-Aware critique [Anonymous, 2025] argues current RCA benchmarks are oversimplified and that published scores overstate real capability. We treat all of these as **external sanity yardsticks only**, never as dependencies, and we adopt their critique as our own Section 6 caveat.

Change / deploy correlation. The single most predictive RCA signal is recent change. The Google SRE Book states that $\sim 70\%$ of outages are due to changes in a live system [Beyer et al., 2016]. The Microsoft Teams incident study [Ghosh et al., 2022] decomposes root-cause categories — Code Bug 27%, Dependency 16.4%, Infra 15.8%, Deployment 13.2%, Config 12.5% — and reports that $\sim 80\%$ of incidents are mitigated *without* a code/config fix, with rollback the #1 mitigation. The classic configuration-error study [Yin et al., 2011] found 16.1–47.3% of misconfigurations cause full unavailability or severe degradation. These motivate first-class **Deploy/Change** graph nodes and an onset-window change term in our ranker (FR-SG-25, FR-SG-26).

Agent failure modes. The MAST failure taxonomy [Cemri et al., 2025] (310 ITBench traces) finds that incorrect self-verification, premature termination, memory loss, and reasoning-action mismatch account for $\sim 94\%$ of agent failures, with the prescriptive lessons: never let the LLM grade its own homework; require hard tool evidence before exit; enforce termination via an FSM outside the model. Our gate and shadow-to-active discipline are direct responses; the verification-gate FSM and hypothesis-state machine are detailed in companion papers in this series.

3 Method / Architecture

3.1 The deterministic replay harness

The canonical eval is `RcaEvalHarness` (`src/DebugAgent/StackGraph/Eval/RcaEvalHarness.cs`), an offline .NET test/CLI entrypoint — **not** a new in-cluster service — that replays the labelled ground-truth set through the real RCA path (`StackGraphRcaService` $\rightarrow$ `RootCauseScorer/OnsetWindowChangeScorer` over a seeded FalkorDB) and emits a deterministic AC@1 + MTTR/TTM scorecard, per-incident and aggregate. It is the source-of-truth realisation of FR-SG-23 (ADR-036 D1).

The data flow is: `ground-truth-incidents.json` (9 labelled incidents) is read by the harness, which seeds a clean FalkorDB graph (`bluerobin_stack_eval`), runs the real `StackGraphRcaService` ranking path, and produces an `EvalScorecard` (AC@1 + MTTR per incident). That scorecard is fed, together with the committed `scorecard-baseline.json`, to the `RcaEvalRegressionGate`: within tolerance, the merge is allowed; on regression, the gate exits 1 and the merge is blocked.

Two arms. The harness deliberately exposes two scoring paths:

- **ReplayAsync** — the **scorer-only** arm. Each incident’s labelled root cause is seeded as the unambiguous top-1 candidate (`RootCauseCriticality` = 0.9, `RootCauseErrorRate` = 0.9), so AC@1 = 1.0 *by construction*. This is a **regression sentinel**, not an accuracy measurement: it proves that a code change did not break the plumbing that maps a seeded subgraph to the correct top-1.
- **ReplayLivePathAsync** — the **live-path** arm. It runs onset-from-alert and injects **distractor candidates**: sibling services the symptom also calls, with `DistractorErrorRate` = 0.55 (non-trivial but below the root cause’s 0.9) and `DistractorCriticality` = 0.95 (deliberately *higher* topology than the root cause’s 0.9 — a topology trap). Because a distractor

outranks the true cause on the topology term, $AC@1 < 1.0$ is now achievable. This is the honest production-path accuracy gate.

The distractor design is the methodological crux. Without it, the eval is gameable: any ranker that trusts topology blindly still scores 1.0. With it, a candidate that boosts topology weight at the expense of the error-rate / change-onset signal will mis-localise GT incidents and drop $AC@1$, failing the gate. The trap is exactly the failure mode the literature warns about — topology criticality is necessary but not sufficient; the discriminating signal is the per-service error-rate plus the in-window change.

3.2 The ground-truth incident set

The labelled set lives in `src/DebugAgent/eval/ground-truth-incidents.json` (`schemaVersion: 1, windowSeconds: 1800`). It is **seeded only from existing faults** — **no ChaosMesh (RISK-1)**: the `bluerobin-e2e` synthetic-test family, the `bluerobin-infra` synthetic-registry scenarios (the OBS-9 login break is canonical), and past `agent_runs` incidents replayed offline. There are 9 labelled incidents spanning ≥ 3 fault classes (auth/login, data/datastore, deploy/config) so the set is not trivially gameable by a single-class heuristic.

Table 1: The labelled ground-truth incident set (9 incidents, ≥ 3 fault classes).

ID	faultClass	expectedRootCause	changeInduced
GT-01-login-break	auth/login	authelia	false
GT-02-rca-gateway-down-floor	deploy/config	mcp-gateway	false
GT-03-bad-deploy-archives-api	deploy/config	archives-api	true
GT-04-falkordb-down	data/datastore	falkordb	false
GT-05-postgres-meshed-reset	data/datastore	postgres	false
GT-06-upload-thumbnail-fail	data/datastore	archives-worker-infra	false
GT-07-bad-commit-web	deploy/config	bluerobin-web	true
GT-08-rag-chat-down	auth/login	authelia	false
GT-09-ollama-embeddings-down	data/datastore	ollama	false

Two incidents (GT-03, GT-07) are explicitly **change-induced** — each carries a `Deploy/Change` node inside the 30-minute onset window. They are the regression test for change-correlation: every active-mode ablation must keep $AC@1 = 1.0$ on GT-03 and GT-07 (no true-positive suppression of the change signal). The same JSON is the single source for both the eval set and the simulation/chaos catalog (`OQ-CATALOG-SHARE`), so the labelled truth and the on-demand simulation harness can never drift apart.

3.3 Metrics and the oracle

$AC@1$ is an exact-string oracle: the produced top-1 candidate’s natural key must equal the labelled `ExpectedRootCauseNaturalKey`. There is **no LLM-judge and no human in the per-commit path**. `MTTR/TTM` is the replay wall-clock from onset-replay-start to the pipeline returning its conclusion (a `Stopwatch` around `InvestigateAsync`) — a time-to-mitigate *proxy*, not production `MTTR`. With a single labelled cause per incident scored at top-1, `Avg@k` and `MRR` reduce to $AC@1$ here; they are computed but carry no extra information until multi-candidate labels land (ADR-036 D1).

Table 2: Metrics computed by the harness and their gating role.

Metric	Definition (this harness)	Role
AC@1	1 iff top-1 candidate key = labelled root-cause key	Primary; gated first
MTTR/TTM	replay wall-clock onset $\rightarrow$ conclusion	Secondary; gated, 30 s tolerance
Avg@k, MRR	reduce to AC@1 (single-label)	Reserved for multi-label future

3.4 The regression gate and tolerances

The gate is `RcaEvalRegressionGate.Evaluate`, run by `RcaEvalGateRunner` with committed default tolerances:

```
public static readonly GateTolerance DefaultTolerance =
    new(AcAt1Drop: 0.02, MtrrSecondsIncrease: 30.0);
```

AC@1 (higher-is-better) is checked **first**: a candidate scorecard whose aggregate AC@1 drops more than 0.02 below the committed baseline fails with exit code 1 and the regressed metric named “AC@1”. MTTR (lower-is-better) is checked second: a rise of more than 30 s fails as “MTTR”. A drift *within* tolerance passes — the 30 s MTTR band deliberately absorbs machine-dependent replay-clock jitter (sub-second MTTR values are explicitly not asserted exactly). The committed baseline `_comment` states the contract verbatim: it “FAILS any candidate that regresses AC@1 below 0.98 or inflates MTTR by > 30 s.”

This gate is the load-bearing artefact of the whole programme: it is what makes every *later* claim in this paper series — graph blame-propagation, BARO onset detection, evidence-chain confidence, episodic memory — *measurable*. None of them can be merged if they regress localisation on the ground-truth set.

3.5 Change correlation as the primary signal

Because $\sim 70\%$ of outages follow a change [Beyer et al., 2016], the ranker treats in-window change as a first-class term. The pre-LLM scoring blend is a re-normalised additive blend over present terms. The base 4-term blend (`OnsetWindowChangeScorer`) and the 5-term BARO blend (`BaroBlendScorer`) are shown in Table 3.

Table 3: Pre-LLM scoring-blend weights. w_{topo} topology criticality, w_{tele} telemetry/error-rate, w_{hist} historical prior, w_{chg} in-window change, w_{baro} BARO onset.

Blend	w_{topo}	w_{tele}	w_{hist}	w_{chg}	w_{baro}	Σ
4-term onset (<code>OnsetWindowChangeScorer</code>)	0.40	0.30	0.15	0.15	—	1.00
5-term BARO (<code>BaroBlendScorer</code>)	0.35	0.25	0.15	0.15	0.10	1.00
Fail-open floor (no chg/baro)	0.50	0.35	0.15	—	—	1.00

A candidate is boosted by w_{chg} when its owning service had a `Deploy/Change` node in the 30-minute onset window $([T - W, T], W = 1800 \text{ s})$. When the change or BARO term is absent, the blend re-normalises over the present terms (the fail-open floor), and that collapse is byte-for-byte identical to the prior baseline — so a missing signal can never *introduce* a regression. The distractor arm (Section 3.1) is precisely what forces this term to earn its weight: a distractor with higher topology but no in-window change must not outrank a true change-induced cause, or GT-03/GT-07 fail.

3.6 Shadow-to-active rollout

New reasoning components do not light up on merge. They ship behind `RcaLiveActivation.Mode` (`src/DebugAgent/Rca/RcaLiveActivation.cs`, ADR-039 D1):

```
public enum RcaLiveActivationMode
{
    Shadow = 0, // default(enum) - compute + log "would-do", never govern
    Active,     // the gate / BARO / memory drive the real decision
}
```

In **Shadow** (the default), the verification gate and BARO scorer compute and log what they *would* do, but the legacy path still governs; the eval collapses byte-for-byte to the prior baseline. Promotion to **Active** is an explicit operator config change (ConfigMap-backed), not a code default. This gives a clean ablation structure: for each increment we assert (a) **active**: no regression versus baseline; (b) **shadow**: collapses to baseline exactly (the “feature off” control); (c) **onset true-positives preserved**: GT-03/GT-07 stay at AC@1 = 1.0. The same shadow-first flag governs the evidence-chain soft-cap (`RcaEvidenceCap.Mode`) and incident-memory recall (`RcaMemory.Mode`).

This is the methodology that makes the system **safe to ship**: a new component is observed against the gate in shadow before it is ever allowed to change an operator-facing conclusion.

4 Implementation

The harness is pure .NET 10. Determinism (NFR-SG-5) is engineered, not hoped for:

- A clean graph per run (`MATCH (n) DETACH DELETE n`), no live-graph reads, empty history priors seeded.
- Stable incident and distractor ordering (JSON order), so the per-commit gate is not flaky.
- An isolated FalkorDB key, `bluerobin_stack_eval` — never the live `bluerobin_stack` graph.
- Fixed onset epochs (1700000000+) so scoring has no wall-clock dependence; only MTTR is wall-clock, and it is absorbed by the 30 s tolerance.
- A reproducibility assertion: two consecutive runs must agree on aggregate AC@1 to 10^{-9} and produce identical incident ordering.

CI wiring. `debug-agent-pipeline.yml` runs the deterministic gate per-commit on every push to main: it spins up `falkordb/falkordb:v4.4.1` on an isolated port, runs `dotnet run ... -- --job=eval` against `ground-truth-incidents.json` and `scorecard-baseline.json`, and fails the build on regression. StackGraph and lifecycle integration suites run via Testcontainers against real FalkorDB and `postgres:16` — there is no DB mock. The optional LLM-judge (`RcaEvalJudgeOptions`, default OFF in CI, hard `PerRunTokenCap: 50_000`) runs only on the nightly `eval-nightly.yml` cron and **never gates merges** — this is what keeps the judge inside the 20 EUR/month LLM ceiling.

This work is governed by FEAT-027, specifically FR-SG-23 (the eval harness + AC@1/MTTR scorecard), FR-SG-24 (the regression gate + bounded LLM-judge), FR-SG-25 (Deploy/Change nodes), and FR-SG-27 (Grafana deploy annotations), under NFR-SG-5 (deterministic reproducibility) and ADR-036 (eval harness + change correlation) / ADR-039 (Phase-B shadow→active live activation).

The ~50 EUR/month angle. The entire programme runs inside a hard platform ceiling of ~50 EUR/month: a single Hetzner CX43 worker (~17.5 EUR/mo), Cloudflare R2 and Tunnel on free tiers, local Ollama embeddings, and a hard 20 EUR/month LLM token ceiling (COST-7, with fail-closed at 100% budget). Two design choices fall directly out of that ceiling. First, the per-commit oracle is deterministic and LLM-free, so the gate that runs on *every* commit costs zero tokens. Second, the eval reuses the *already-deployed* FalkorDB instance under an isolated key rather than standing up a benchmarking cluster — a live ChaosMesh-style fault-injection benchmark over a separate Kubernetes environment is simply not affordable, which is why the ground-truth set is seeded from existing faults. The cost constraint is not a footnote; it is the reason the methodology looks the way it does.

5 Evaluation

The committed scorecards are the baselines the gate protects. The **scorer-only** baseline (`eval/scorecard-baseline`) is $AC@1 = 1.0$ by construction across all 9 incidents — a regression sentinel. The **live-path** baseline (`src/DebugAgent/eval/scorecard-live-path-baseline.json`) is the honest production-path number: even with distractors injected, all 9 incidents still localise at top-1, so the measured live-path $AC@1$ is also 1.0 on this set.

Table 4: Committed baseline scorecards (the baselines the per-commit gate protects).

Arm	AC@1	MTTR (s)	Incidents	Avg@k	MRR	Interpretation
Scorer-only (ReplayAsync)	1.0	0.0017	9	1.0	1.0	By construction; sentinel
Live-path (ReplayLivePathAsync)	1.0	0.0032	9	1.0	1.0	Honest path; distractors

The scorer-only arm is a regression sentinel, **not** an accuracy claim. The live-path arm is the honest production path: distractors are injected, and $AC@1 < 1.0$ is achievable. Per-incident MTTR ranges 0.0007–0.0073 s (scorer-only) and 0.0016–0.0126 s (live-path) — sub-second, machine-dependent, and **not asserted exactly**.

How to read these numbers honestly. The live-path arm proves the ranker is not fooled by the topology trap on *these* nine incidents — a real result, because $AC@1 = 1.0$ there is *not* guaranteed by the harness; the distractor with higher criticality could legitimately win. But nine self-seeded incidents is a tiny, low-diversity set, and a perfect score on it says far more about the absence of regressions than about absolute capability. For external context: the best agent on OpenRCA reaches ~11.3% exact-match [Xu et al., 2025] and the best frontier model on ITBench-AA reaches ~47% precision@full-recall [Artificial Analysis and IBM Research, 2026] on far harder, larger, adversarial sets. Our 1.0 is **not** comparable to those numbers and we do not claim it is. The harness’s own source calls the by-construction arm “the §12 oversimplified-benchmark caveat” — an in-code acknowledgement that the planted 1.0 is plumbing, not accuracy.

What the evaluation *does* establish is the property we actually need: every increment (verification gate + BARO, evidence soft-cap, memory recall) passes the gate in both active and shadow modes, and the change-induced incidents GT-03/GT-07 retain $AC@1 = 1.0$ under every active-mode ablation. That is a no-regression guarantee, not a capability record — which is exactly the claim an eval-first methodology should make.

6 Limitations & Threats to Validity

We state these plainly; they are not hedges, they are the honest scope of the work.

- **Small, self-seeded ground truth (construct + external validity).** Nine incidents, seeded by the authors from existing faults, with no ChaosMesh or live fault injection. The set spans ≥ 3 fault classes specifically to avoid single-class gaming, but it is small and low-diversity. $AC@1 = 1.0$ on it does not generalise to production RCA difficulty.
- **Author-labelled oracle (internal validity).** The expected root cause for each incident is an author judgement. Exact-string matching is unambiguous *given* the label, but the labels themselves are not independently adjudicated.
- **MTTR is a replay-clock proxy (construct validity).** Reported MTTR is sub-second offline wall-clock around `InvestigateAsync`, not production fire-to-resolved MTTR. Real life-cycle MTTD/MTTA/MTTR p50/p95 are computed separately from live data (`GetKpisEndpoint`, SL0-15/SL0-16) but no measured production values are committed here.
- **By-construction $AC@1 = 1.0$ (presentation risk).** The scorer-only arm is a sentinel; reported naively it overstates capability. We mitigate by always reporting the live-path arm alongside it and by labelling the scorer arm as plumbing.
- **No external-baseline comparison.** OpenRCA / ITBench-AA / AIOpsLab are external yardsticks, never dependencies; we do not run them, so our numbers are not cross-comparable. This is deliberate (cost and reproducibility) but it means we cannot claim relative standing against published systems.
- **Benchmark-oversimplification critique applies to us too.** The Fault-Propagation-Aware critique [Anonymous, 2025] — that simple methods match SOTA on easy benchmarks — applies with full force to a 9-incident self-seeded set. We adopt the critique rather than resist it.
- **Advisory-only by design.** A human always merges; nothing auto-remediates (ADR-030). We position this as matching the state of the art given the $\sim 11\text{--}47\%$ autonomous ceiling [Xu et al., 2025, Artificial Analysis and IBM Research, 2026], not as a temporary limitation.

7 Conclusion

LLM-only RCA loops have no native sense of correctness or regression, which makes them dangerous to evolve and hard to ship responsibly. The BlueRobin Debug Agent addresses this not with a bigger model but with a discipline: a deterministic offline replay harness over a labelled ground-truth set, a per-commit regression gate with explicit $AC@1$ /MTTR tolerances, an honest distractor-injected live-path arm that makes $AC@1 < 1.0$ achievable, change-correlation as the primary signal, and a shadow-to-active rollout that observes every new reasoning component before it is allowed to govern. None of this produces a record accuracy number — by design, and we say so. What it produces is the precondition for everything else in this series: a way to add graph reasoning, change-point detection, evidence-grounded confidence and episodic memory to an LLM-in-the-loop RCA agent and *know*, on every commit, that one did not make it worse — all inside a ~ 50 EUR/month homelab budget. The honest contribution is the gate, not the score.

References

- Anonymous. Rethinking the evaluation of root cause analysis: A fault-propagation-aware benchmark, 2025. URL <https://arxiv.org/abs/2510.04711>. arXiv:2510.04711; argues current RCA benchmarks are oversimplified and that published scores overstate real capability.
- Artificial Analysis and IBM Research. ITBench-AA: An agentic sre root-cause-analysis leaderboard. IBM Research blog, 2026. URL <https://huggingface.co/blog/ibm-research/itbench-aa>. Best frontier model reaches roughly 47% precision@full-recall; all frontier models below 50%.
- Betsy Beyer, Chris Jones, Jennifer Petoff, and Niall Richard Murphy. *Site Reliability Engineering: How Google Runs Production Systems*. O’Reilly Media, 2016. URL <https://sre.google/sre-book/introduction/>. The Google SRE Book; states that roughly 70% of outages are due to changes in a live system.
- Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Matei Zaharia, Kannan Ramchandran, Ion Stoica, et al. Why do multi-agent LLM systems fail? a failure taxonomy (MAST). IBM Research / UC Berkeley; 310 ITBench traces, 2025. URL <https://huggingface.co/blog/ibm-research/itbenchandmast>. Finds incorrect self-verification, premature termination, memory loss, and reasoning-action mismatch account for roughly 94% of agent failures.
- Yinfang Chen, Manish Shetty, Gagan Somashekar, Minghua Ma, Yogesh Simmhan, Jonathan Mace, Chetan Bansal, Rujia Wang, and Saravan Rajmohan. AIOpsLab: A holistic framework to evaluate ai agents for enabling autonomous clouds, 2025. URL <https://arxiv.org/abs/2501.06706>. arXiv:2501.06706.
- Datadog. Bits ai sre: Deeper reasoning for automated investigations. Datadog Engineering blog, 2026. URL <https://www.datadoghq.com/blog/bits-ai-sre-deeper-reasoning/>. Scores against a labeled set with an LLM-judge and surfaces an agent trace; hypothesis tree with validated / invalidated / inconclusive states.
- Eval-Driven Development. Eval-driven development: The eval comes first. evaldriven.org, 2025. URL <https://evaldriven.org/>.
- Supriyo Ghosh, Manish Shetty, Chetan Bansal, and Suman Nath. How to fight production incidents? an empirical study on a large-scale cloud service. In *Proceedings of the 13th Symposium on Cloud Computing (SoCC ’22)*. ACM, 2022. URL <https://www.microsoft.com/en-us/research/wp-content/uploads/2022/09/3542929.3563482.pdf>. Empirical study of incidents in Microsoft Teams.
- Saurabh Jha, Rohan Arora, Yuji Watanabe, Takumi Yanagawa, Yinfang Chen, Jackson Clark, Bhavya Bhavya, Mudit Verma, Harshit Kumar, Hirokuni Kitahara, et al. ITBench: Evaluating ai agents across diverse real-world it automation tasks, 2025. URL <https://arxiv.org/abs/2502.05352>. arXiv:2502.05352; introduces the Normalized Topology-Aware Metric; reports MTTD 72 s / MTTR 282 s.
- Luan Pham, Hongyu Ha, and Huong Zhang. RCAEval: A benchmark for root cause analysis of microservice systems with telemetry data, 2024. URL <https://arxiv.org/abs/2412.17015>. arXiv:2412.17015; 735 cases, 11 fault types, 15 baselines; standardizes the AC@k / Avg@k / MRR metric family.

- Robusta and Microsoft. HolmesGPT: Ai agent for on-call engineers — evaluations as per-commit regression tests. CNCF Sandbox; runs 150+ evals on every commit, 2025. URL <https://holmesgpt.dev/latest/development/evaluations/>.
- Junjielong Xu, Qinan Zhang, Zhiqing Zhong, Shilin He, Chaoyun Zhang, Qingwei Lin, Dan Pei, Pinjia He, Dongmei Zhang, and Qi Zhang. OpenRCA: Can large language models locate the root cause of software failures? In *Proceedings of the 13th International Conference on Learning Representations (ICLR '25)*, 2025. URL <https://openreview.net/pdf?id=M4qNIzQYpd>. OpenReview M4qNIzQYpd; best agent reaches roughly 11.3% exact-match over 335 cases.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models, 2022. URL <https://arxiv.org/abs/2210.03629>. arXiv:2210.03629.
- Zuoning Yin, Xiao Ma, Jing Zheng, Yuanyuan Zhou, Lakshmi N. Bairavasundaram, and Shankar Pasupathy. An empirical study on configuration errors in commercial and open source systems. In *Proceedings of the 23rd ACM Symposium on Operating Systems Principles (SOSP '11)*. ACM, 2011. URL <https://www.sigops.org/s/conferences/sosp/2011/current/2011-Cascais/printable/12-yin.pdf>.